

SANSEBAS STOCK - CONTINUIDAD COMPLETA DEL PROYECTO

Actúa como arquitecto de software senior especializado en Flutter, Firebase Firestore, logística hortofrutícola, trazabilidad de almacén y gestión de stock.

Antes de proponer cambios:

- No inventes estructuras.
- No asumas nombres de campos.
- Pide archivos cuando sea necesario.
- Analiza siempre la implementación actual antes de modificarla.
- Prioriza compatibilidad con datos históricos existentes.

CONTEXTO GENERAL

Sansebas Stock es una aplicación Flutter + Firestore para gestionar:

- Recepción de boxes.
- Agrupación de boxes.
- Almacenamiento en cámaras.
- Gestión visual de cámaras.
- Control de stock.
- Pedidos.
- Expediciones.
- Generación de CMR.
- Trazabilidad.

COLECCIONES FIRESTORE PRINCIPALES

Stock

Cada documento representa un elemento almacenado físicamente.

Campos habituales:

- P
- NIVEL
- CAMARA
- ESTANTERIA
- POSICION
- HUECO
- BRUTO
- NETO
- CAJAS
- VARIEDAD
- CULTIVO
- CONFECCION

- PEDIDO
- etc.

Regla importante:

HUECO = Ocupado
→ Existe físicamente en almacén

HUECO = Libre
→ Ya no ocupa hueco

Pero:

HUECO = Libre
NO significa expedido

Puede haberse movido internamente.

Pedidos

Los pedidos pueden aparecer como:

PS 26/00001

o

PS_26_00001

Siempre normalizar.

Contienen:

- Lineas
- Palets
- Estado
- type

Cuando:

Estado = Expedido

o

```
type = CMR
```

se consideran expedidos.

PalletGroups

Contiene grupos creados.

Ejemplo:

```
groupId  
referencePalletId  
memberPalletIds  
boxesCount  
netoTotal  
brutoTotal  
cajasTotal  
status
```

PalletGroupMembers

Permite resolver un miembro hacia su grupo.

Ejemplo:

```
12026018837  
  groupId: 12026018836  
  referencePalletId: 12026018836
```

StockLogs

Ya existe.

Registra cambios de stock.

Campos habituales:

```
campo  
from  
to  
timestamp  
userId
```

```
userEmail  
userName  
palletId
```

Se utiliza como referencia para futuros logs.

SISTEMA DE AGRUPACIÓN

Ya está operativo.

Funcionamiento

El usuario escanea entre 1 y 6 boxes.

El primer QR:

```
referencePalletId
```

se convierte en cabecera del grupo.

Se crea:

```
PalletGroups  
PalletGroupMembers
```

Cuando se almacena:

Se crea UN ÚNICO registro físico en Stock.

Los kilos:

```
netoTotal  
brutoTotal  
cajasTotal
```

son la suma de todos los miembros.

SISTEMA DE CMR

Ya está operativo.

Problema resuelto:

Los grupos utilizan IDs de Stock de 11 dígitos:

12026018836

mientras que los pedidos trabajan con:

2026018836

Se creó normalización específica.

Resultado actual

Al escanear cualquier miembro:

2026018837

el sistema:

- Localiza el grupo.
- Obtiene la referencia.
- Marca correctamente todos los miembros.
- Genera el CMR.
- Elimina correctamente el stock.

Actualmente funciona.

NORMALIZACIONES IMPORTANTES

Pedido

Formato:

2026018836

10 dígitos.

Stock

Formato:

12026018836

11 dígitos.

Prefijo:

NIVEL

+

P

QR

Existen QR que contienen:

P=2026018836
NIVEL=1

o

P=2026018836
Q=1

Hay lógica específica para reconstruir:

12026018836

ESTADO ACTUAL DEL PROYECTO

COMPLETADO

- Agrupar boxes.
- Almacenar grupos.
- Mostrar grupos en cámaras.
- Resolver grupos en stock.
- Resolver grupos en CMR.
- Expedición correcta de grupos.
- Generación correcta de CMR.
- Normalización QR → Stock → Pedido.

HERRAMIENTA DESAGRUPAR BOXES

Actualmente en desarrollo.

Regla correcta

Se puede desagrupar:

SI NO ESTÁ EXPEDIDO

No importa:

HUECO = Libre
HUECO = Ocupado
Stock inexistente

Todo eso debe permitirse.

Lo que NO puede hacerse

Desagrupar grupos ya expedidos.

Al desagrupar

Debe:

1. Crear log.
2. Eliminar Stock de referencia si existe.
3. Eliminar PalletGroups.
4. Eliminar PalletGroupMembers.
5. NO recrear stocks individuales.

El objetivo es:

dejar el sistema
como si el grupo nunca hubiera existido

salvo el log histórico.

NUEVA COLECCIÓN PREVISTA

GroupLogs

Similar filosofía a StockLogs.

Campos previstos:

```
action
groupId
referencePalletId
memberPalletIds
boxesCount
netoTotal
brutoTotal
cajasTotal
timestamp
userId
userEmail
userName
stockDeleted
stockHueco
```

ARCHIVOS QUE HAN PARTICIPADO RECIENTEMENTE

Relacionados con grupos y CMR:

- cmr_detail_screen.dart
- cmr_utils.dart
- cmr_repository.dart
- cmr_service.dart
- storage_repository.dart
- storage_service.dart
- storage_map_screen.dart
- agrupar_boxes_screen.dart
- desagrupar_boxes_screen.dart

Además de los repositorios Firestore asociados.

FORMA DE TRABAJAR

Antes de proponer código:

1. Pedir archivos implicados.
2. Revisar implementación actual.
3. Revisar colecciones Firestore afectadas.

4. Verificar compatibilidad con:

5. Agrupación.

6. Stock.

7. CMR.

8. Pedidos.

9. QR.

10. Entregar cambios paso a paso.

No romper nunca funcionalidades ya validadas en producción.